# ASSIGNMENT- 09

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

**Subject: Database Management System**

**Topic: Advanced SQL Queries Using Joins and Subqueries**

By
**Name:** MERIN THOMAS
**Reg. No.:** 2511021240062

**Faculty Name:** RENITA JOY
**Faculty Signature:**_____

**Department of Computer Application**
**Alliance University**
**Chandapura - Anekal Main Road, Anekal**
**Bengaluru - 562 106**
**August 2026**

# 1. Aim

To implement SQL views, indexes, and transactions in the Paranormal Investigation Agency Management System to simplify data retrieval, improve query performance, and maintain database consistency.

# 2. Introduction

Views, indexes, and transactions are important features of a relational database management system.

- **Views** provide virtual tables based on SQL queries and simplify access to frequently required information.
- **Indexes** improve the speed of data retrieval by helping the database locate records efficiently.
- **Transactions** group multiple database operations into a single logical unit and help maintain data consistency.

These features can help a paranormal investigation agency manage cases, investigators, evidence, and payments more efficiently.

# 3. Implementation of SQL Views

## 3.1 Create a View for Case Details

```
CREATE VIEW Case_Details_View AS

SELECT C.Case_id,

    CL.name AS Client_Name,

    L.addess AS Location,

    CC.category_name AS Case_Category,

    C.Investigation_date,

    C.Case_status,

    C.Urgency_level

FROM Cases C

INNER JOIN Clients CL

  ON C.Client_id = CL.client_id

INNER JOIN Locations L
```

```
    ON C.Location_id = L.location_id
INNER JOIN Case_Categories CC
    ON C.Category_id = CC.Category_id;
```

**Explanation:** This view combines case, client, location, and category information into one virtual table.

**To display the view:**

```
SELECT * FROM Case_Details_View;
```

**Expected output:** A list of cases showing client names, locations, categories, investigation dates, statuses, and urgency levels.

## 3.2 Create a View for Pending Payments

```
CREATE VIEW Pending_Payments_View AS
SELECT P.Payment_id,
    P.Case_id,
    CL.name AS Client_Name,
    P.Amount,
    P.Payment_mode,
    P.Payment_status
FROM Payments P
INNER JOIN Cases C
    ON P.Case_id = C.Case_id
INNER JOIN Clients CL
    ON C.Client_id = CL.client_id
WHERE P.Payment_status = 'Pending';
```

**Explanation:** This view displays payment records that are still pending, along with the associated client names.

**To display the view:**

```
SELECT * FROM Pending_Payments_View;
```

**Expected output:** Pending payment records for Cases 102 and 105, based on the sample data from Assignment 7.

### 3.3 Create a View for Investigator Assignments

```
CREATE VIEW Investigator_Assignment_View AS

SELECT CI.Case_id,

    I.name AS Investigator_Name,

    I.speciliazation AS Specialization,

    CI.Role_in_case

FROM Case_Investigator CI

INNER JOIN Investigators I

  ON CI.Investigator_id = I.investigation_id;
```

**Explanation:** This view displays investigators assigned to each case, along with their specialization and role.

**To display the view:**

```
SELECT * FROM Investigator_Assignment_View;
```

**Expected output:** Investigator assignment records from the Case_Investigator table.

## 4. Implementation of Indexes

An index is a database object that helps retrieve records more efficiently. Indexes are especially useful for columns frequently used in filtering, sorting, and joining operations.

### 4.1 Create an Index on Case Status

```
CREATE INDEX idx_case_status
ON Cases (Case_status);
```

**Explanation:** This index can help queries that filter cases by status.

**Example query:**

```
SELECT *
FROM Cases
WHERE Case_status = 'Completed';
```

## 4.2 Create an Index on Investigation Date

```
CREATE INDEX idx_investigation_date
ON Cases (Investigation_date);
```

**Explanation:** This index can help retrieve cases based on investigation dates.

**Example query:**

```
SELECT *
FROM Cases
WHERE Investigation_date >= '2026-03-01'
ORDER BY Investigation_date;
```

## 4.3 Create an Index on Payment Status

```
CREATE INDEX idx_payment_status
ON Payments (Payment_status);
```

**Explanation:** This index can help retrieve payment records by their status.

**Example query:**

```
SELECT *

FROM Payments

WHERE Payment_status = 'Pending';
```

**Note:** Indexes improve some read operations but require additional storage and can slow down inserts, updates, and deletes. The actual performance improvement depends on the dataset and query execution plan.

# 5. Implementation of Transactions

A transaction is a group of SQL operations treated as a single logical unit. MySQL supports transaction control through commands such as `START TRANSACTION`, `COMMIT`, and `ROLLBACK` when using a transactional storage engine such as InnoDB.

## 5.1 Example of a Successful Transaction

The following example records a payment for an existing case.

```
START TRANSACTION;


INSERT INTO Payments

(Payment_id, Case_id, Amount, Payment_mode,

 Payment_status, Payment_date)

VALUES

(6, 103, 5000.00, 'UPI', 'Paid', '2026-04-10');


COMMIT;
```

**Explanation:**

1. `START TRANSACTION` begins a transaction.
2. `INSERT` adds a new payment record.
3. `COMMIT` permanently saves the change.

**Verification query:**

```
SELECT *

FROM Payments

WHERE Payment_id = 6;
```

**Expected output:** The newly inserted payment record, provided the transaction completed successfully.

## 5.2 Example of Rollback

The following example demonstrates how to cancel a change before it is committed.

```
START TRANSACTION;


UPDATE Cases

SET Case_status = 'Completed'

WHERE Case_id = 102;


ROLLBACK;
```

**Explanation:**

1. The transaction begins.
2. The status of Case 102 is temporarily changed.
3. `ROLLBACK` cancels the uncommitted change.

**Verification query:**

```
SELECT Case_id, Case_status

FROM Cases

WHERE Case_id = 102;
```

**Expected output:** The case status remains `In Progress`, assuming it had that status before the transaction and no other operation changed it.

**Important:** Run these transaction examples separately. Do not execute `COMMIT` after the rollback example if you intend to demonstrate the cancellation of that transaction.

# 6. Verification of Views and Indexes

## 6.1 Display All Views

```
SHOW FULL TABLES

WHERE Table_type = 'VIEW';
```

**Explanation:** Displays the views created in the current database.

## 6.2 Display Indexes on the Cases Table

```
SHOW INDEX FROM Cases;
```

**Explanation:** Displays the indexes associated with the Cases table.

## 6.3 Display Indexes on the Payments Table

```
SHOW INDEX FROM Payments;
```

**Explanation:** Displays the indexes associated with the Payments table.

# 7. Advantages

## Views

- Simplify complex SQL queries.
- Present related information in a single virtual table.
- Help restrict access to selected columns.
- Improve readability and reusability of queries.

## Indexes

- Improve data retrieval for suitable queries.
- Support filtering, sorting, and joins.
- Help retrieve frequently searched records efficiently.

## Transactions

- Maintain consistency during database operations.
- Allow changes to be saved with `COMMIT`.
- Allow uncommitted changes to be cancelled with `ROLLBACK`.
- Help prevent incomplete changes when related operations must succeed together.

# 8. Result

SQL views were designed to simplify access to case details, pending payments, and investigator assignments. Indexes were created on frequently queried columns, and transaction examples demonstrated how to save or cancel database changes.

# 9. Conclusion

This assignment demonstrates the use of views, indexes, and transactions in the Paranormal Investigation Agency Management System. Views simplify data access, indexes can improve retrieval performance, and transactions help maintain data consistency. Together, these features support efficient and reliable database management.